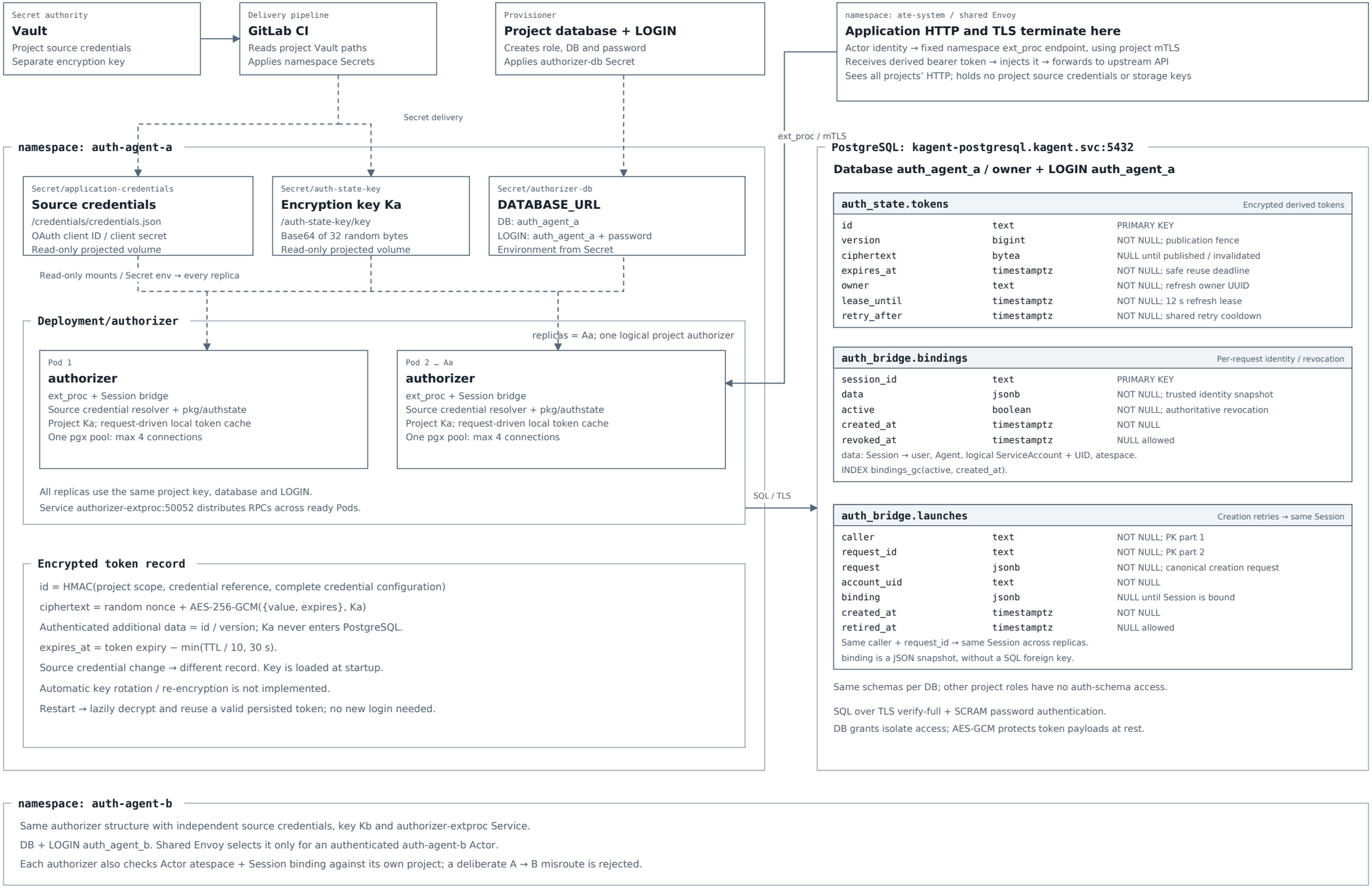
Secret authority
Vault
Project source credentials
Separate encryption key
Delivery pipeline
GitLab CI
Reads project Vault paths
Applies namespace Secrets
Provisioner
Project database + LOGIN
Creates role, DB and password
Applies authorizer-db Secret
namespace: ate-system / shared Envoy
Application HTTP and TLS terminate here
Actor identity → fixed namespace ext_proc endpoint, using project mTLS
Receives derived bearer token → injects it → forwards to upstream API
Sees all projects' HTTP; holds no project source credentials or storage keys
Secret delivery
ext_proc / mTLS
namespace: auth-agent-a
Secret/application-credentials
Source credentials
/credentials/credentials.json
OAuth client ID / client secret
Read-only projected volume
Secret/auth-state-key
Encryption key Ka
/auth-state-key/key
Base64 of 32 random bytes
Read-only projected volume
Secret/authorizer-db
DATABASE_URL
DB: auth_agent_a
LOGIN: auth_agent_a + password
Environment from Secret
Read-only mounts / Secret env → every replica
Deployment/authorizer
replicas = Aa; one logical project authorizer
Pod 1
authorizer
ext_proc + Session bridge
Source credential resolver + pkg/authstate
Project Ka; request-driven local token cache
One pgx pool: max 4 connections
Pod 2 … Aa
authorizer
ext_proc + Session bridge
Source credential resolver + pkg/authstate
Project Ka; request-driven local token cache
One pgx pool: max 4 connections
All replicas use the same project key, database and LOGIN.
Service authorizer-extproc:50052 distributes RPCs across ready Pods.
SQL / TLS
Encrypted token record
id = HMAC(project scope, credential reference, complete credential configuration)
ciphertext = random nonce + AES-256-GCM({value, expires}, Ka)
Authenticated additional data = id / version; Ka never enters PostgreSQL.
expires_at = token expiry − min(TTL / 10, 30 s).
Source credential change → different record. Key is loaded at startup.
Automatic key rotation / re-encryption is not implemented.
Restart → lazily decrypt and reuse a valid persisted token; no new login needed.
PostgreSQL: kagent-postgresql.kagent.svc:5432
Database auth_agent_a / owner + LOGIN auth_agent_a
auth_state.tokens
Encrypted derived tokens
id text PRIMARY KEY
version bigint NOT NULL; publication fence
ciphertext bytea NULL until published / invalidated
expires_at timestamptz NOT NULL; safe reuse deadline
owner text NOT NULL; refresh owner UUID
lease_until timestamptz NOT NULL; 12 s refresh lease
retry_after timestamptz NOT NULL; shared retry cooldown
auth_bridge.bindings
Per-request identity / revocation
session_id text PRIMARY KEY
data jsonb NOT NULL; trusted identity snapshot
active boolean NOT NULL; authoritative revocation
created_at timestamptz NOT NULL
revoked_at timestamptz NULL allowed
data: Session → user, Agent, logical ServiceAccount + UID, atespace.
INDEX bindings_gc(active, created_at).
auth_bridge.launches
Creation retries → same Session
caller text NOT NULL; PK part 1
request_id text NOT NULL; PK part 2
request jsonb NOT NULL; canonical creation request
account_uid text NOT NULL
binding jsonb NULL until Session is bound
created_at timestamptz NOT NULL
retired_at timestamptz NULL allowed
Same caller + request_id → same Session across replicas.
binding is a JSON snapshot, without a SQL foreign key.
Same schemas per DB; other project roles have no auth-schema access.
SQL over TLS verify-full + SCRAM password authentication.
DB grants isolate access; AES-GCM protects token payloads at rest.
namespace: auth-agent-b
Same authorizer structure with independent source credentials, key Kb and authorizer-extproc Service.
DB + LOGIN auth_agent_b. Shared Envoy selects it only for an authenticated auth-agent-b Actor.
Each authorizer also checks Actor atespace + Session binding against its own project; a deliberate A → B misroute is rejected.